**DARSHAN PRAJAPATI**
**MCA-1**
**ROLL NO: 24**
**SUBMISSION DATE: 21/11/2025**

**OOCP PRACTICAL ASSIGNMENT-3**

**// Inheritance**

**// 1) WAP to create base class Book having int id and char name as data members**
**// and respective functionality, show following types of inheritance and display**
**// the details of each kind of books, also calculate the total no of each type of**
**// books in proper format.**
**// Simple inheritance with derived class Sales**
**// Hierarchical inheritance with derived classes academics and thrillers**
**// Show use of constructor and destructor in above examples of inheritance.**

```
#include <iostream>
#include <cstring>
using namespace std;

// Base Class
class Book
{
protected:
   int id;
   char name[50];
   static int totalBooks;

public:
   Book()
   {
      totalBooks++;
      cout << "\n[Book Constructor Called]";
   }

   virtual ~Book()
   {
      cout << "\n[Book Destructor Called]";
```

```
    }

    void getData()
    {
        cout << "\nEnter Book ID: ";
        cin >> id;
        cout << "Enter Book Name: ";
        cin.ignore();
        cin.getline(name, 50);
    }

    void displayData()
    {
        cout << "\nBook ID: " << id;
        cout << "\nBook Name: " << name;
    }

    static void showTotalBooks()
    {
        cout << "\nTotal Books (All Types): " << totalBooks;
    }
};

int Book::totalBooks = 0;

// ---------------- SIMPLE INHERITANCE ----------------

class Sales : public Book
{
    float price;
    static int totalSalesBooks;

public:
    Sales()
    {
        totalSalesBooks++;
```

```
        cout << "\n[Sales Constructor Called]";
    }

    ~Sales()
    {
        cout << "\n[Sales Destructor Called]";
    }

    void getData()
    {
        Book::getData();
        cout << "Enter Book Price: ";
        cin >> price;
    }

    void displayData()
    {
        cout << "\n--- Sales Book Details ---";
        Book::displayData();
        cout << "\nPrice: " << price;
    }

    static void showTotalSalesBooks()
    {
        cout << "\nTotal Sales Books: " << totalSalesBooks;
    }
};

int Sales::totalSalesBooks = 0;

// ---------------- HIERARCHICAL INHERITANCE ----------------

class Academics : public Book
{
    char subject[30];
    static int totalAcademicBooks;
```

```
public:
    Academics()
    {
        totalAcademicBooks++;
        cout << "\n[Academics Constructor Called]";
    }

    ~Academics()
    {
        cout << "\n[Academics Destructor Called]";
    }

    void getData()
    {
        Book::getData();
        cout << "Enter Subject: ";
        cin.ignore();
        cin.getline(subject, 30);
    }

    void displayData()
    {
        cout << "\n--- Academic Book Details ---";
        Book::displayData();
        cout << "\nSubject: " << subject;
    }

    static void showTotalAcademicBooks()
    {
        cout << "\nTotal Academic Books: " << totalAcademicBooks;
    }
};

int Academics::totalAcademicBooks = 0;
```

```
class Thrillers : public Book
{
    char author[30];
    static int totalThrillerBooks;

public:
    Thrillers()
    {
        totalThrillerBooks++;
        cout << "\n[Thrillers Constructor Called]";
    }

    ~Thrillers()
    {
        cout << "\n[Thrillers Destructor Called]";
    }

    void getData()
    {
        Book::getData();
        cout << "Enter Author Name: ";
        cin.ignore();
        cin.getline(author, 30);
    }

    void displayData()
    {
        cout << "\n--- Thriller Book Details ---";
        Book::displayData();
        cout << "\nAuthor: " << author;
    }

    static void showTotalThrillerBooks()
    {
        cout << "\nTotal Thriller Books: " << totalThrillerBooks;
    }
```

```
};

int Thrillers::totalThrillerBooks = 0;

// ---------------- MAIN FUNCTION ----------------

int main()
{
    cout << "=== SIMPLE INHERITANCE ===";
    Sales s1;
    s1.getData();
    s1.displayData();

    cout << "\n\n=== HIERARCHICAL INHERITANCE ===";
    Academics a1;
    a1.getData();
    a1.displayData();

    Thrillers t1;
    t1.getData();
    t1.displayData();

    cout << "\n\n=== TOTAL COUNTS ===";
    Book::showTotalBooks();
    Sales::showTotalSalesBooks();
    Academics::showTotalAcademicBooks();
    Thrillers::showTotalThrillerBooks();

    cout << "\n\nProgram End - Destructors will be called automatically.\n";

    return 0;
}

// Output:
// === SIMPLE INHERITANCE ===
// [Book Constructor Called]
```

```
// [Sales Constructor Called]
// Enter Book ID: 101
// Enter Book Name: Osho
// Enter Book Price: 1000

// --- Sales Book Details ---
// Book ID: 101
// Book Name: Osho
// Price: 1000

// === HIERARCHICAL INHERITANCE ===
// [Book Constructor Called]
// [Academics Constructor Called]
// Enter Book ID: 102
// Enter Book Name: Puppu
// Enter Subject: comedy

// --- Academic Book Details ---
// Book ID: 102
// Book Name: Puppu
// Subject: omedy
// [Book Constructor Called]
// [Thrillers Constructor Called]
// Enter Book ID: 103
// Enter Book Name: Fitnesss
// Enter Author Name: Dp

// --- Thriller Book Details ---
// Book ID: 103
// Book Name: Fitnesss
// Author: p

// === TOTAL COUNTS ===
// Total Books (All Types): 3
// Total Sales Books: 1
// Total Academic Books: 1
```

```
// Total Thriller Books: 1

// Program End - Destructors will be called automatically.

// [Thrillers Destructor Called]
// [Book Destructor Called]
// [Academics Destructor Called]
// [Book Destructor Called]
// [Sales Destructor Called]
// [Book Destructor Called]
```

**// 2) WAP to create student having data members (rollno, name, stream) as base**
**// class. Derive class subject with marks of 5 subjects and apply respective**
**// functionality. Calculate final result and display details of each student from**
**// derived class. (multilevel inheritance).**

```
#include <iostream>
using namespace std;

class Student
{
   int rollno;
   string name;
   string stream;

public:
   Student()
   {
      cout << "Base constructor called ..." << endl;
   }
   virtual ~Student()
   {
      cout << "Base Destructor called" << endl;
   }
   void getData()
   {
```

```
        cout << "\nEnter Roll No: ";
        cin >> rollno;
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Stream: ";
        cin >> stream;
    }
    void displayData()
    {
        cout << "\n--- Student Details ---" << endl;
        cout << "Roll No: " << rollno << "\nName: " << name << "\nStream: " << stream << endl;
    }
};

class Subject : public Student
{
protected:
    int marks[5];

public:
    Subject()
    {
        cout << "Subject class constructor called ..." << endl;
    }
    ~Subject()
    {
        cout << "Subject class Destructor called ..." << endl;
    }
    void getData()
    {
        Student::getData();
        cout << "\nEnter marks of 5 subjects:" << endl;
        for (int i = 0; i < 5; i++)
        {
            cout << "Subject " << i + 1 << ": ";
            cin >> marks[i];
```

```
        }
    }
    void displayData()
    {
        Student::displayData();
        cout << "\n--- Marks Details ---" << endl;
        for (int i = 0; i < 5; i++)
        {
            cout << "Subject " << i + 1 << ": " << marks[i] << endl;
        }
    }
};

class Result : public Subject
{
    int total = 0;
    float percentage;

public:
    Result()
    {
        cout << "Result class constructor called ..." << endl;
    }
    ~Result()
    {
        cout << "Result class destructor called ..." << endl;
    }
    void calResult()
    {
        total = 0;
        for (int i = 0; i < 5; i++)
        {
            total += marks[i];
        }
        percentage = (static_cast<float>(total) / 500) * 100;
    }
```

```
    void displayData()
    {
        calResult();
        Subject::displayData();
        cout << "\n--- Final Result ---" << endl;
        cout << "Total Marks: " << total << "/500" << endl;
        cout << "Percentage: " << percentage << "%" << endl;
    }
};

int main()
{
    Result s1;
    s1.getData();
    s1.displayData();
    return 0;
}

// output:

// Base constructor called ...
// Subject class constructor called ...
// Result class constructor called ...

// Enter Roll No: 101
// Enter Name: dp
// Enter Stream: commerce

// Enter marks of 5 subjects:
// Subject 1: 83
// Subject 2: 32
// Subject 3: 83
// Subject 4: 23
// Subject 5: 33

// --- Student Details ---
```

```
// Roll No: 101
// Name: dp
// Stream: commerce

// --- Marks Details ---
// Subject 1: 83
// Subject 2: 32
// Subject 3: 83
// Subject 4: 23
// Subject 5: 33

// --- Final Result ---
// Total Marks: 254/500
// Percentage: 50.8%
// Result class destructor called ...
// Subject class Destructor called ...
// Base Destructor called
```

**// 3) WAP to create Class Person, Employee and Department and Apply respective**
**// functionalities using inheritance.**

```
#include <iostream>
using namespace std;

class Person
{
protected:
   string name;
   int age;

public:
   Person()
   {
      cout << "\n[Person Constructor Called]" << endl;
```

```
    }

    virtual ~Person()
    {
        cout << "[Person Destructor Called]" << endl;
    }
    void getPersonaData()
    {
        cout << "Enter name:-";
        cin >> name;
        cout << "Enter age:-";
        cin >> age;
    }
    void displayPersonData()
    {
        cout << "\n --- Person Details --- \n";
        cout << "Name: " << name << "\n Age:" << age << endl;
    }
};

class Employee : public Person
{
protected:
    int empId;
    float salary;

public:
    Employee()
    {
        cout << "[Employee Constructor Called]" << endl;
    }

    ~Employee()
    {
        cout << "[Employee Destructor Called]" << endl;
    }
```

```
    void getEmployeeData()
    {
        getPersonaData();
        cout << "Enter Employee ID: ";
        cin >> empId;
        cout << "Enter Salary: ";
        cin >> salary;
    }

    void displayEmployeeData()
    {
        displayPersonData();
        cout << "\n--- Employee Details ---" << endl;
        cout << "Employee ID: " << empId << "\nSalary: " << salary << endl;
    }
};

class Department : public Employee
{
private:
    string deptName;
    string designation;

public:
    Department()
    {
        cout << "[Department Constructor Called]" << endl;
    }

    ~Department()
    {
        cout << "[Department Destructor Called]" << endl;
    }

    void getDepartmentData()
    {
```

```
        getEmployeeData();
        cout << "Enter Department Name: ";
        cin >> deptName;
        cout << "Enter Designation: ";
        cin >> designation;
    }

    void displayDepartmentData()
    {
        displayEmployeeData();
        cout << "\n--- Department Details ---" << endl;
        cout << "Department: " << deptName << "\nDesignation: " << designation << endl;
    }
};

int main()
{
    Department d1;

    cout << "\n Enter Department Employee Details:\n";
    d1.getDepartmentData();

    cout << "\n Display Details: \n";
    d1.displayDepartmentData();

    return 0;
}

// Output:
// [Person Constructor Called]
// [Employee Constructor Called]
// [Department Constructor Called]

//  Enter Department Employee Details:
// Enter name:-Darshan
// Enter age:-82
```

```
// Enter Employee ID: 101
// Enter Salary: 82300
// Enter Department Name: IT
// Enter Designation: senior web devloper

//  Display Details:

//  --- Person Details ---
// Name: Darshan
//  Age:82

// --- Employee Details ---
// Employee ID: 101
// Salary: 82300

// --- Department Details ---
// Department: IT
// Designation: senior
// [Department Destructor Called]
// [Employee Destructor Called]
// [Person Destructor Called]
```

**// 4) An educational institution wishes to maintain a database of its employees.**
**// The database is divided into a number of classes whose hierarchical**
**// relationships are shown in fig-1. The figure also shows the minimum**
**// information required for each class. Specify all the classes and define function**
**// to create the database and retrieve individual information as and when**
**// required. Write parameterized constructor for each class in the hierarchy.**

```
#include <iostream>
using namespace std;

// Base class: Staff
class Staff
{
   int code;
```

```
    string name;

public:
    Staff(int c, string n) : code(c), name(n)
    {
        cout << "\n[Staff Constructor Called...]";
    }
    virtual ~Staff()
    {
        cout << "\n[Staff Destructor Called...]";
    }
    virtual void DisplayData()
    {
        cout << "\n\n-- Details of Staff ---";
        cout << "\nCode of Staff: " << code;
        cout << "\nName of Staff: " << name;
    }
};

// Derived class: Teacher
class Teacher : public Staff
{
    string subject, publication;

public:
    Teacher(int c, string n, string s, string p)
        : Staff(c, n), subject(s), publication(p)
    {
        cout << "\n[Teacher Constructor Called...]";
    }
    ~Teacher()
    {
        cout << "\n[Teacher Destructor Called...]";
    }
    void DisplayData() override
    {
```

```
        Staff::DisplayData();
        cout << "\n-- Details of Teacher ---";
        cout << "\nSubject: " << subject;
        cout << "\nPublication: " << publication;
    }
};

// Derived class: Typist
class Typist : public Staff
{
protected:
    float speed;

public:
    Typist(int c, string n, float s) : Staff(c, n), speed(s)
    {
        cout << "\n[Typist Constructor Called...]";
    }
    ~Typist()
    {
        cout << "\n[Typist Destructor Called...]";
    }
    void DisplayData() override
    {
        Staff::DisplayData();
        cout << "\n-- Details of Typist ---";
        cout << "\nSpeed: " << speed;
    }
};

// Derived class: Officer
class Officer : public Staff
{
    char grade;

public:
```

```
    Officer(int c, string n, char g) : Staff(c, n), grade(g)
    {
        cout << "\n[Officer Constructor Called...]";
    }
    ~Officer()
    {
        cout << "\n[Officer Destructor Called...]";
    }
    void DisplayData() override
    {
        Staff::DisplayData();
        cout << "\n-- Details of Officer ---";
        cout << "\nGrade: " << grade;
    }
};

// Derived class: Regular (from Typist)
class Regular : public Typist
{
public:
    Regular(int c, string n, float s) : Typist(c, n, s)
    {
        cout << "\n[Regular Constructor Called...]";
    }
    ~Regular()
    {
        cout << "\n[Regular Destructor Called...]";
    }
    void DisplayData() override
    {
        Typist::DisplayData();
        cout << "\n-- Details of Regular Typist ---";
    }
};

// Derived class: Casual (from Typist)
```

```
class Casual : public Typist
{
    int dailywage;

public:
    Casual(int c, string n, float s, int da) : Typist(c, n, s), dailywage(da)
    {
        cout << "\n[Casual Constructor Called...]";
    }
    ~Casual()
    {
        cout << "\n[Casual Destructor Called...]";
    }
    void DisplayData() override
    {
        Typist::DisplayData();
        cout << "\n-- Details of Casual Typist ---";
        cout << "\nDaily Wage: " << dailywage;
    }
};

int main()
{
    cout << "\n========== Staff Database ==========\n";

    Staff s1(101, "Darshan");
    s1.DisplayData();

    Teacher t1(102, "Riya", "Maths", "Pearson");
    t1.DisplayData();

    Typist t2(103, "Anil", 85);
    t2.DisplayData();

    Officer o1(104, "Kavita", 'A');
    o1.DisplayData();
```

```
    Regular r1(105, "Rahul", 90);
    r1.DisplayData();

    Casual c1(106, "Priya", 75, 600);
    c1.DisplayData();

    cout << "\n\n===================================\n";
    return 0;
}

// output:
// ========== Staff Database ==========

// [Staff Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 101
// Name of Staff: Darshan
// [Staff Constructor Called...]
// [Teacher Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 102
// Name of Staff: Riya
// -- Details of Teacher ---
// Subject: Maths
// Publication: Pearson
// [Staff Constructor Called...]
// [Typist Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 103
// Name of Staff: Anil
// -- Details of Typist ---
// Speed: 85
```

```
// [Staff Constructor Called...]
// [Officer Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 104
// Name of Staff: Kavita
// -- Details of Officer ---
// Grade: A
// [Staff Constructor Called...]
// [Typist Constructor Called...]
// [Regular Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 105
// Name of Staff: Rahul
// -- Details of Typist ---
// Speed: 90
// -- Details of Regular Typist ---
// [Staff Constructor Called...]
// [Typist Constructor Called...]
// [Casual Constructor Called...]

// -- Details of Staff ---
// Code of Staff: 106
// Name of Staff: Priya
// -- Details of Typist ---
// Speed: 75
// -- Details of Casual Typist ---
// Daily Wage: 600

// ====================================

// [Casual Destructor Called...]
// [Typist Destructor Called...]
// [Staff Destructor Called...]
// [Regular Destructor Called...]
```

```
// [Typist Destructor Called...]
// [Staff Destructor Called...]
// [Officer Destructor Called...]
// [Staff Destructor Called...]
// [Typist Destructor Called...]
// [Staff Destructor Called...]
// [Teacher Destructor Called...]
// [Staff Destructor Called...]
// [Staff Destructor Called...]
```

**// 5) Create a class student from which the classes test and sports are derived.**
**// The class student has the name and rollno of the student. The class test has**
**// the marks of the internal test and the sports class has the marks of the sports**
**// test. The class student contains a virtual function display() which are**
**// implemented in the classes test and sports. Write a program which will take**
**// relative information and display it using pointer of the base class.**

```
#include <iostream>
#include <cstring>
using namespace std;

// ---------------- BASE CLASS ----------------
class Student
{
protected:
   char name[50];
   int rollno;

public:
   void getData()
   {
      cout << "\nEnter Name: ";
      cin.ignore();
      cin.getline(name, 50);
      cout << "Enter Roll No: ";
```

```
        cin >> rollno;
    }

    // Virtual function to be overridden in derived classes
    virtual void display()
    {
        cout << "\n--- Student Information ---";
        cout << "\nName: " << name;
        cout << "\nRoll No: " << rollno;
    }

    // Virtual destructor (good practice)
    virtual ~Student() {}
};

// ---------------- DERIVED CLASS 1 ----------------
class Test : public Student
{
    float testMarks;

public:
    void getData()
    {
        Student::getData();
        cout << "Enter Internal Test Marks: ";
        cin >> testMarks;
    }

    void display() override
    {
        cout << "\n--- Test Details ---";
        cout << "\nName: " << name;
        cout << "\nRoll No: " << rollno;
        cout << "\nInternal Test Marks: " << testMarks;
    }
};
```

```
// ---------------- DERIVED CLASS 2 ----------------
class Sports : public Student
{
   float sportsMarks;

public:
   void getData()
   {
      Student::getData();
      cout << "Enter Sports Marks: ";
      cin >> sportsMarks;
   }

   void display() override
   {
      cout << "\n--- Sports Details ---";
      cout << "\nName: " << name;
      cout << "\nRoll No: " << rollno;
      cout << "\nSports Marks: " << sportsMarks;
   }
};

// ---------------- MAIN FUNCTION ----------------
int main()
{
   Student *ptr; // Base class pointer

   Test t;
   cout << "\nEnter Test Student Details:";
   t.getData();

   Sports s;
   cout << "\nEnter Sports Student Details:";
   s.getData();
```

```
    cout << "\n\n--- DISPLAYING DETAILS USING BASE CLASS POINTER ---";

    ptr = &t;       // Pointing to Test object
    ptr->display(); // Calls Test::display()

    ptr = &s;       // Pointing to Sports object
    ptr->display(); // Calls Sports::display()

    cout << "\n\nProgram ended successfully.\n";
    return 0;
}

// output:

// Enter Test Student Details:
// Enter Name: Darshna
// Enter Roll No: 101
// Enter Internal Test Marks: 88

// Enter Sports Student Details:
// Enter Name: dp
// Enter Roll No: 83
// Enter Sports Marks: 78

// --- DISPLAYING DETAILS USING BASE CLASS POINTER ---
// --- Test Details ---
// Name: arshna
// Roll No: 101
// Internal Test Marks: 88
// --- Sports Details ---
// Name: dp
// Roll No: 83
// Sports Marks: 78

// Program ended successfully.
```

**// 6) Define a class Figure and use inheritance to define the classes Triangle,Square, Circle, and Rectangle. Provide a container class Frame to contain any number of figures, possibly overlapped in position. Provide operations for drawing frames on the screen and for inserting, modifying, and deleting frames and the contents of frames. (Actual drawing is not required).**

```
//      figure
// circle squre triangle rectangle

#include <iostream>
using namespace std;

class Figure
{
public:
   virtual ~Figure() = default; // <-- virtual destructor (safe)
   virtual void draw()
   {
      cout << "Drawing base class figure\n";
   };
};

class Triangle : public Figure
{
public:
   void draw() override
   {
      cout << "Drawing Triangle\n";
   }
};

class Square : public Figure
{
public:
   void draw() override
   {
      cout << "Drawing Square\n";
```

```
    }
};

class Circle : public Figure
{
public:
    void draw() override
    {
        cout << "Drawing Circle\n";
    }
};

class Rectangle : public Figure
{
public:
    void draw() override
    {
        cout << "Drawing Rectangle\n";
    }
};

int main()
{
    Figure *f1 = nullptr; // initialize pointer
    Circle c1;
    Square s1;
    Rectangle r1;
    Triangle t1;
    int choice;

    while (true)
    {
        cout << "\nEnter the shape you want to draw, -1 to stop drawing:\n"
            << "1. Triangle\t2. Circle\t3. Square\t4. Rectangle\n";
        if (!(cin >> choice))
        {
```

```
        cout << "Invalid input. Exiting.\n";
        break;
    }

    if (choice == 1)
    {
        f1 = &t1;
    }
    else if (choice == 2)
    {
        f1 = &c1;
    }
    else if (choice == 3)
    {
        f1 = &s1;
    }
    else if (choice == 4)
    {
        f1 = &r1;
    }
    else if (choice == -1)
    {
        cout << "Bye bye!\n";
        break;
    }
    else
    {
        cout << "Invalid choice!\n";
        continue;
    }

    if (f1)
        f1->draw();
}

return 0;
```

```
}

// Output:

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// 1
// Drawing Triangle

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// 2
// Drawing Circle

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// 3
// Drawing Square

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// 4
// Drawing Rectangle

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// 41
// Invalid choice!

// Enter the shape you want to draw, -1 to stop drawing:
// 1. Triangle     2. Circle      3. Square      4. Rectangle
// -1
// Bye bye!
```

**// 7) Define a class Student. Inherit this class into MCAStudent and**

**// NonMCAStudent. Also inherit it into Local and NonLocal students. Multiple**

**// inherit LocalMCAStudent from Local and MCAStudent. Define five instances**

**// of LocalMCAStudent with a constructor, assuming that all classes have a**

**// constructor.**

**// student=>local nonlocal mca nonmca**

**// local mca=> localMcaStudent**

```
#include <iostream>
#include <string>
using namespace std;

// Base class
class Student
{
protected:
   string name;
   int rollNo;

public:
   Student(string n, int r)
   {
      name = n;
      rollNo = r;
      cout << "Student constructor called for " << name << endl;
   }

   void displayStudent()
   {
      cout << "Name: " << name << ", Roll No: " << rollNo << endl;
   }

   ~Student()
   {
      cout << "Student destructor called for " << name << endl;
   }
```

```
};

// Derived from Student
class MCAStudent : virtual public Student
{
protected:
   string specialization;

public:
   MCAStudent(string n, int r, string spec) : Student(n, r)
   {
      specialization = spec;
      cout << "MCAStudent constructor called for " << name << endl;
   }

   void displayMCA()
   {
      cout << "Specialization: " << specialization << endl;
   }

   ~MCAStudent()
   {
      cout << "MCAStudent destructor called for " << name << endl;
   }
};

// Derived from Student
class NonMCAStudent : public Student
{
protected:
   string courseName;

public:
   NonMCAStudent(string n, int r, string course) : Student(n, r)
   {
      courseName = course;
```

```
        cout << "NonMCAStudent constructor called for " << name << endl;
    }

    void displayNonMCA()
    {
        cout << "Course Name: " << courseName << endl;
    }

    ~NonMCAStudent()
    {
        cout << "NonMCAStudent destructor called for " << name << endl;
    }
};

// Another derived class from Student
class Local : virtual public Student
{
protected:
    string city;

public:
    Local(string n, int r, string c) : Student(n, r)
    {
        city = c;
        cout << "Local constructor called for " << name << endl;
    }

    void displayLocal()
    {
        cout << "City: " << city << endl;
    }

    ~Local()
    {
        cout << "Local destructor called for " << name << endl;
    }
```

```
};

// Another derived class from Student
class NonLocal : public Student
{
protected:
   string hostelName;

public:
   NonLocal(string n, int r, string hostel) : Student(n, r)
   {
      hostelName = hostel;
      cout << "NonLocal constructor called for " << name << endl;
   }

   void displayNonLocal()
   {
      cout << "Hostel: " << hostelName << endl;
   }

   ~NonLocal()
   {
      cout << "NonLocal destructor called for " << name << endl;
   }
};

// Multiple Inheritance
class LocalMCAStudent : public Local, public MCAStudent
{
public:
   LocalMCAStudent(string n, int r, string c, string spec)
      : Student(n, r), Local(n, r, c), MCAStudent(n, r, spec)
   {
      cout << "LocalMCAStudent constructor called for " << n << endl;
   }
```

```
    void display()
    {
        cout << "\n--- Local MCA Student Details ---" << endl;
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollNo << endl;
        cout << "City: " << city << endl;
        cout << "Specialization: " << specialization << endl;
    }

    ~LocalMCAStudent()
    {
        cout << "LocalMCAStudent destructor called for " << name << endl;
    }
};

// Main function
int main()
{
    cout << "Creating Local MCA Students...\n";

    LocalMCAStudent s1("Darshan", 101, "Ahmedabad", "AI-ML");
    LocalMCAStudent s2("Raj", 102, "Vadodara", "Data Science");
    LocalMCAStudent s3("Priya", 103, "Surat", "Cyber Security");
    LocalMCAStudent s4("Karan", 104, "Rajkot", "Web Development");
    LocalMCAStudent s5("Neha", 105, "Bhavnagar", "Cloud Computing");

    cout << "\nDisplaying all students:\n";
    s1.display();
    s2.display();
    s3.display();
    s4.display();
    s5.display();

    cout << "\nProgram Ended...\n";
    return 0;
}
```

// // output:

// Creating Local MCA Students...

// Student constructor called for Darshan

// Local constructor called for Darshan

// MCAStudent constructor called for Darshan

// LocalMCAStudent constructor called for Darshan

// Student constructor called for Raj

// Local constructor called for Raj

// MCAStudent constructor called for Raj

// LocalMCAStudent constructor called for Raj

// Student constructor called for Priya

// Local constructor called for Priya

// MCAStudent constructor called for Priya

// LocalMCAStudent constructor called for Priya

// Student constructor called for Karan

// Local constructor called for Karan

// MCAStudent constructor called for Karan

// LocalMCAStudent constructor called for Karan

// Student constructor called for Neha

// Local constructor called for Neha

// MCAStudent constructor called for Neha

// LocalMCAStudent constructor called for Neha

// Displaying all students:

// --- Local MCA Student Details ---

// Name: Darshan

// Roll No: 101

// City: Ahmedabad

// Specialization: AI-ML

// --- Local MCA Student Details ---

// Name: Raj

// Roll No: 102

```
// City: Vadodara
// Specialization: Data Science

// --- Local MCA Student Details ---
// Name: Priya
// Roll No: 103
// City: Surat
// Specialization: Cyber Security

// --- Local MCA Student Details ---
// Name: Karan
// Roll No: 104
// City: Rajkot
// Specialization: Web Development

// --- Local MCA Student Details ---
// Name: Neha
// Roll No: 105
// City: Bhavnagar
// Specialization: Cloud Computing

// Program Ended...
// LocalMCAStudent destructor called for Neha
// MCAStudent destructor called for Neha
// Local destructor called for Neha
// Student destructor called for Neha
// LocalMCAStudent destructor called for Karan
// MCAStudent destructor called for Karan
// Local destructor called for Karan
// Student destructor called for Karan
// LocalMCAStudent destructor called for Priya
// MCAStudent destructor called for Priya
// Local destructor called for Priya
// Student destructor called for Priya
// LocalMCAStudent destructor called for Raj
// MCAStudent destructor called for Raj
```

```
// Local destructor called for Raj
// Student destructor called for Raj
// LocalMCAStudent destructor called for Darshan
// MCAStudent destructor called for Darshan
// Local destructor called for Darshan
// Student destructor called for Darshan
```

**// 8) Define a class Faculty. It contains the following attributes:**

**// (a) Name of the faculty**

**// (b) Qualification of the faculty**

**// (c) Subjects the faculty can teach**

**// Inherit the Faculty class into a regular faculty, who**

**// (a) is available full time (no consulting time is specified)**

**// (b) teaches at least three subjects**

**// (c) is part of the institute alone**

**// Inherit the Faculty class into visiting faculty, who**

**// (a) is available only on two to three days (consulting time is also to be**

**// specified)**

**// (b) teaches a single subject**

**// (c) is part of some other institute**

**// Provide virtual functions for reading and writing class objects of the classes**

**// given above.**

```
#include <iostream>
#include <string>
using namespace std;

// Base class
class Faculty
{
protected:
   string name;
   string qualification;
   string subjects[10]; // Fixed-size array for subjects
   int subjectCount;
```

```
public:
    Faculty()
    {
        subjectCount = 0;
    }

    // Virtual functions
    virtual void read()
    {
        cout << "Enter Faculty Name: ";
        getline(cin, name);
        cout << "Enter Qualification: ";
        getline(cin, qualification);

        cout << "Enter number of subjects faculty can teach: ";
        cin >> subjectCount;
        cin.ignore();

        for (int i = 0; i < subjectCount; i++)
        {
            cout << "Enter subject " << i + 1 << ": ";
            getline(cin, subjects[i]);
        }
    }

    virtual void display()
    {
        cout << "\nFaculty Name: " << name << endl;
        cout << "Qualification: " << qualification << endl;
        cout << "Subjects: ";
        for (int i = 0; i < subjectCount; i++)
        {
            cout << subjects[i];
            if (i != subjectCount - 1)
                cout << ", ";
        }
```

```
        cout << endl;
    }

    virtual ~Faculty() {}
};

// Derived class for Regular Faculty
class RegularFaculty : public Faculty
{
public:
    void read() override
    {
        cout << "\n--- Enter Regular Faculty Details ---\n";
        Faculty::read();

        if (subjectCount < 3)
        {
            cout << "Regular faculty must teach at least 3 subjects.\n";
            while (subjectCount < 3)
            {
                cout << "Enter additional subject " << subjectCount + 1 << ": ";
                getline(cin, subjects[subjectCount]);
                subjectCount++;
            }
        }
    }

    void display() override
    {
        cout << "\n--- Regular Faculty Details ---";
        Faculty::display();
        cout << "Availability: Full Time\n";
        cout << "Institute Type: Internal (Part of the institute alone)\n";
    }
};
```

```
// Derived class for Visiting Faculty
class VisitingFaculty : public Faculty
{
    int consultingDays;
    string consultingTime;
    string otherInstitute;

public:
    void read() override
    {
        cout << "\n--- Enter Visiting Faculty Details ---\n";
        cout << "Enter Faculty Name: ";
        getline(cin, name);
        cout << "Enter Qualification: ";
        getline(cin, qualification);

        subjectCount = 1;
        cout << "Enter the subject taught: ";
        getline(cin, subjects[0]);

        cout << "Enter number of consulting days (2 or 3): ";
        cin >> consultingDays;
        cin.ignore();

        cout << "Enter consulting time: ";
        getline(cin, consultingTime);

        cout << "Enter other institute name: ";
        getline(cin, otherInstitute);
    }

    void display() override
    {
        cout << "\n--- Visiting Faculty Details ---";
        Faculty::display();
        cout << "Consulting Days: " << consultingDays << endl;
```

```
        cout << "Consulting Time: " << consultingTime << endl;
        cout << "Other Institute: " << otherInstitute << endl;
    }
};

// Main function
int main()
{
    Faculty *f1, *f2;

    f1 = new RegularFaculty();
    f2 = new VisitingFaculty();

    cout << "Enter details for Regular Faculty:\n";
    f1->read();

    cout << "\nEnter details for Visiting Faculty:\n";
    f2->read();

    cout << "\n============================\n";
    cout << "Displaying Faculty Details:\n";
    cout << "============================\n";

    f1->display();
    f2->display();

    delete f1;
    delete f2;

    return 0;
}

// OUTPUT

// Enter details for Regular Faculty:
```

```
// --- Enter Regular Faculty Details ---
// Enter Faculty Name: DP SIR
// Enter Qualification: B.C.A.
// Enter number of subjects faculty can teach: 2
// Enter subject 1: DBMS
// Enter subject 2: OOCP
// Regular faculty must teach at least 3 subjects.
// Enter additional subject 3: DBMS PART 2

// Enter details for Visiting Faculty:

// --- Enter Visiting Faculty Details ---
// Enter Faculty Name: HARDIK PANDYA
// Enter Qualification: 12TH PASS
// Enter the subject taught: P.T.
// Enter number of consulting days (2 or 3): 3
// Enter consulting time: 3 HOURS
// Enter other institute name: M.S. DHONI ACEDAMI

// ============================
// Displaying Faculty Details:
// ============================

// --- Regular Faculty Details ---
// Faculty Name: DP SIR
// Qualification: B.C.A.
// Subjects: DBMS, OOCP, DBMS PART 2
// Availability: Full Time
// Institute Type: Internal (Part of the institute alone)

// --- Visiting Faculty Details ---
// Faculty Name: HARDIK PANDYA
// Qualification: 12TH PASS
// Subjects: P.T.
// Consulting Days: 3
// Consulting Time: 3 HOURS
```

// Other Institute: M.S. DHONI ACEDAMI

**// 9) Create a base class called shape. Use this class to store two double type values**
**// that could be used to compute the area of figures. Derive three specific classes**
**// called triangle, rectangle and circle from the base shape. Add to the base class,**
**// a member function get_data( ) to initialize base class data members and**
**// another member function display_area( ) to compute and display the area of**
**// figures. Make display_area( ) as a virtual function and redefine this function in**
**// derived classes to suit their requirements. Using these three classes design a**
**// program that will accept dimensions of a triangle or rectangle interactively and**
**// store it in one array . After having read all the input display the area of all the**
**// figures whose area has been read in the program. Remember the two values**
**// given as input will be treated as lengths of two sides in the case of rectangle**
**// and as base and height in case of triangle. In case of circle only one value**
**// should be accepted which will be taken as the radius and the default value of**
**// the next parameter should be 0.**

```
#include <iostream>
using namespace std;

// class shape
class Shape
{
protected:
   double v1, v2;

public:
   virtual void get_data()
   {
      cin >> v1 >> v2;
   }
   virtual void display() = 0;
   virtual ~Shape() {};
};

// class Triangle
```

```
class Triangle : public Shape
{
public:
   void get_data() override
   {
      cout << "Enter base and height of triangle:-";
      cin >> v1 >> v2;
   }
   void display() override
   {
      cout << "Triangle Area= " << 0.5 * v1 * v2 << endl;
   }
};
// class Rectangle
class Rectangle : public Shape
{
public:
   void get_data() override
   {
      cout << "Enter Length and width of rectangle:-";
      cin >> v1 >> v2;
   }
   void display() override
   {
      cout << "Rectangle Area= " << 0.5 * v1 * v2 << endl;
   }
};
// class Circle
class Circle : public Shape
{
public:
   void get_data() override
   {
      cout << "Enter Radius of Circle:-";
      cin >> v1;
      v2 = 0;
```

```
    }
    void display() override
    {
        cout << "Circle Area= " << 3.14519 * v1 * v1 << endl;
    }
};
int main()
{
    Shape *arr[10]; // array of shape pointers

    int n;
    cout << "How many shapes? ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        cout << "\n1.Triangle  2.Rectangle  3.Circle\nChoose shape: ";
        int choice;
        cin >> choice;
        if (choice == 1)
        {
            arr[i] = new Triangle;
        }
        else if (choice == 2)
        {
            arr[i] = new Rectangle;
        }
        else
        {
            arr[i] = new Circle;
        }
        arr[i]->get_data(); // take input
    }
    cout << "\n--- Areas ---\n";
    for (int i = 0; i < n; i++)
    {
```

```
        arr[i]->display(); // polymorphic call
    }

    // free memory
    for (int i = 0; i < n; i++)
    {
        delete arr[i];
    }

    return 0;
}

// output:
// How many shapes? 2

// 1.Triangle  2.Rectangle  3.Circle
// Choose shape: 1
// Enter base and height of triangle:-23
// 23

// 1.Triangle  2.Rectangle  3.Circle
// Choose shape: 3
// Enter Radius of Circle:-35

// --- Areas ---
// Triangle Area= 264.5
// Circle Area= 3852.86
```

**// 10) Consider a class student from which the classes test and sports are derived.**
**// The class student has the name and rollno of the student. The class test has**
**// the marks of the internal test and the sports class has the marks of the sports**
**// test. The class student contains a virtual function display() which are**
**// implemented in the classes test and sports. Write a program which will take**
**// relative information and display it using pointer of the base class.**

```
#include <iostream>
#include <string>
using namespace std;

// Base class
class Student
{
protected:
   string name;
   int rollno;

public:
   void read()
   {
      cout << "Enter Student Name: ";
      getline(cin, name);
      cout << "Enter Roll Number: ";
      cin >> rollno;
      cin.ignore();
   }

   // Virtual function
   virtual void display()
   {
      cout << "Name: " << name << ", Roll No: " << rollno << endl;
   }

   virtual ~Student() {}
};

// Derived class: Test
class Test : public Student
{
   float testMarks;

public:
```

```
    void read()
    {
        cout << "\n--- Enter Test Details ---\n";
        Student::read();
        cout << "Enter Internal Test Marks: ";
        cin >> testMarks;
        cin.ignore();
    }

    void display() override
    {
        cout << "\n--- Test Result ---\n";
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollno << endl;
        cout << "Internal Test Marks: " << testMarks << endl;
    }
};

// Derived class: Sports
class Sports : public Student
{
    float sportsMarks;

public:
    void read()
    {
        cout << "\n--- Enter Sports Details ---\n";
        Student::read();
        cout << "Enter Sports Marks: ";
        cin >> sportsMarks;
        cin.ignore();
    }

    void display() override
    {
        cout << "\n--- Sports Result ---\n";
```

```
        cout << "Name: " << name << endl;
        cout << "Roll No: " << rollno << endl;
        cout << "Sports Marks: " << sportsMarks << endl;
    }
};

// Main function
int main()
{
    Student *ptr; // Base class pointer
    Test t;
    Sports s;

    cout << "Enter details for Test Student:\n";
    t.read();

    cout << "\nEnter details for Sports Student:\n";
    s.read();

    cout << "\n==============================\n";
    cout << "Displaying Details using Base Pointer\n";
    cout << "==============================\n";

    ptr = &t;
    ptr->display(); // Calls Test::display()

    ptr = &s;
    ptr->display(); // Calls Sports::display()

    return 0;
}

// output
// Enter details for Test Student:
// --- Enter Test Details ---
// Enter Student Name: Darshan
```

// Enter Roll Number: 101

// Enter Internal Test Marks: 88

// Enter details for Sports Student:

// --- Enter Sports Details ---

// Enter Student Name: Raj

// Enter Roll Number: 102

// Enter Sports Marks: 75

// ==========

// Displaying Details using Base Pointer

// ==========

// --- Test Result ---

// Name: Darshan

// Roll No: 101

// Internal Test Marks: 88

// --- Sports Result ---

// Name: Raj

// Roll No: 102

// Sports Marks: 75

**// 11) ABC publishing company markets both book and audio cassette versions of its**

**// work. Create a class called publication that stores the title( a string) and price(**

**// type float) of a publication. From this class derive two classes : book , which**

**// adds a page count (type int); and tape, which adds playing time in minutes**

**// (type float). Write a main program that reads both book and tape information in**

**// one array. When the user has finished entering data for all books and tapes,**

**// displays the resulting data for all the books and tapes entered. Also count no of**

**// book and cassette entries in the array using runtime identification feature of**

**// C++.**

**// Base class Publication : title string , price float**

**// Derive class Book: pageCount int**

**// Derive class Tape: minutes float**

```
#include <iostream>
using namespace std;

// base class : Publication
class Publication
{
protected:
   string title;
   float price;

public:
   virtual void input()
   {
      cout << "\n Enter the title:- ";
      cin.ignore();
      getline(cin, title);

      cout << "\n Enter the price:- ";
      cin >> price;
   }
   void display()
   {
      cout << "\t Title:" << title << "\t Price:" << price;
   }
};
// derived class: Book
class Book : public Publication
{
private:
   int pageCount;

public:
   void input()
   {
      Publication::input();
```

```
        cout << "Enter the page count: ";
        cin >> pageCount;
    }
    void display()
    {
        Publication::display();
        cout << "\t Page Count :" << pageCount;
    }
};
// derived class: Tape
class Tape : public Publication
{
private:
    float minutes;

public:
    void input()
    {
        Publication::input();
        cout << "Enter the minutes: ";
        cin >> minutes;
    }
    void display()
    {
        Publication::display();
        cout << "\t Audio minutes :" << minutes;
    }
};
int main()
{
    Publication *items[20];
    int n, type;
    int bookCount = 0, tapeCount = 0;
    cout << "How many publication do you want to store? ";
    cin >> n;
```

```
    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter 1 for Book, 2 for Tape: ";
        cin >> type;

        if (type == 1)
            items[i] = new Book;
        else
            items[i] = new Tape;

        items[i]->input();
    }
    cout << "\n\n--- Displaying Publications ---\n";

    for (int i = 0; i < n; i++)
    {
        cout << "\nPublication " << i + 1 << ":\n";
        items[i]->display();

        // Runtime Identification using dynamic_cast
        if (dynamic_cast<Book *>(items[i]) != nullptr)
            bookCount++;
        else if (dynamic_cast<Tape *>(items[i]) != nullptr)
            tapeCount++;
    }

    cout << "\nTotal Books: " << bookCount;
    cout << "\nTotal Tapes: " << tapeCount << endl;

    for (int i = 0; i < n; i++)
        delete items[i];

    return 0;
}

// Output:
```

// How many publication do you want to store? 5

// Enter 1 for Book, 2 for Tape: 1

// Enter the title:- DS Book

// Enter the price:- 120

// Enter the page count: 25

// Enter 1 for Book, 2 for Tape: 2

// Enter the title:- Kishor kumar audio

// Enter the price:- 1200

// Enter the minutes: 820

// Enter 1 for Book, 2 for Tape: 1

// Enter the title:- RDBMS book

// Enter the price:- 839

// Enter the page count: 88

// Enter 1 for Book, 2 for Tape: 2

// Enter the title:- Yo yo honey singh

// Enter the price:- 283

// Enter the minutes: 82

// Enter 1 for Book, 2 for Tape: 1

// Enter the title:- Python for beginner

// Enter the price:- 822

```
// Enter the page count: 833

// --- DISPLAYING PUBLICATIONS ---

// Publication 1:
//        Title:DS Book   Price:120
// Publication 2:
//        Title:Kishor kumar audio       Price:1200
// Publication 3:
//        Title:RDBMS book       Price:839
// Publication 4:
//        Title:Yo yo honey singh        Price:283
// Publication 5:
//        Title:Python for beginner      Price:822
// Total Books: 3
// Total Tapes: 2
```

**// FILES**

**// 12) Write a Program to perform following operation on text file :**

**// 11.1) write content in a text file**

**// 11.2) read content from file**

**// 11.3) count no of word and no of lines in a file**

**// 11.4) copy contents of one file to another file**

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
   string fname1, fname2, line;

   cout << "Enter filename to write: ";
   cin >> fname1;
   cin.ignore();
```

```
ofstream out(fname1);
cout << "Enter text (enter a single . to stop):\n";
while (true)
{
   getline(cin, line);
   if (line == ".")
      break;
   out << line << "\n";
}
out.close();
cout << "Data written.\n\n";

cout << "Reading file: " << fname1 << "\n";
ifstream in(fname1);
while (getline(in, line))
{
   cout << line << "\n";
}
in.close();
cout << "\n";

in.open(fname1);
int lines = 0, words = 0;
string word;

while (getline(in, line))
{
   lines++;
   for (int i = 0; i < line.length(); i++)
   {
      if ((i == 0 || line[i - 1] == ' ') && line[i] != ' ')
         words++;
   }
}
in.close();
```

```
    cout << "Total lines: " << lines << "\n";
    cout << "Total words: " << words << "\n\n";

    cout << "Enter destination filename to copy: ";
    cin >> fname2;

    ifstream src(fname1);
    ofstream dest(fname2);

    while (getline(src, line))
    {
        dest << line << "\n";
    }

    src.close();
    dest.close();

    cout << "File copied to " << fname2 << "\n";

    return 0;
}
// output:
// Enter filename to write: student.txt
// Enter text (enter a single . to stop):
// hello world by student
// .
// Data written.

// Reading file: student.txt
// hello world by student

// Total lines: 1
// Total words: 4

// Enter destination filename to copy: std.txt
```

// File copied to std.txt

**// 13) Write a program to create a file student to store name and marks of 5**

**// students and then display them.**

```
#include <iostream>
#include <fstream>
using namespace std;

int main()
{
   ofstream fout("student.txt");

   string name;
   int marks;

   // Write data of 5 students
   cout << "Enter name and marks of 5 students:\n";
   for (int i = 0; i < 5; i++)
   {
      cout << "Student " << i + 1 << " Name: ";
      cin >> name;
      cout << "Marks: ";
      cin >> marks;

      fout << name << " " << marks << endl;
   }

   fout.close();

   // Read and display
   ifstream fin("student.txt");

   cout << "\n--- Student Details from File ---\n";
   while (fin >> name >> marks)
```

```
    {
        cout << "Name: " << name << ", Marks: " << marks << endl;
    }

    fin.close();
    return 0;
}

// Output:

// Enter name and marks of 5 students:
// Student 1 Name: Meera
// Marks: 78
// Student 2 Name: Mukesh
// Marks: 87
// Student 3 Name: Nilesh
// Marks: 33
// Student 4 Name: Usha
// Marks: 83
// Student 5 Name: Khushi
// Marks: 83

// --- Student Details from File ---
// Name: Meera, Marks: 78
// Name: Mukesh, Marks: 87
// Name: Nilesh, Marks: 33
// Name: Usha, Marks: 83
// Name: Khushi, Marks: 83
```

**// 14) Define a class Result which contains the result of an MCA written test. It should take merit list from a file and display on the screen such that at a time only ten candidates information is printed on the screen. The headings should be displayed using a manipulator. All the information should be aligned with the headings.**

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
```

```
int main()
{
    const string filename = "students.txt";
    ofstream ofs(filename);
    if (!ofs)
    {
        cerr << "Cannot create file.\n";
        return 1;
    }

    cout << "Enter name and marks for 5 students:\n";
    for (int i = 0; i < 5; ++i)
    {
        string name;
        int marks;
        cout << "Student " << i + 1 << " name: ";
        getline(cin, name);
        cout << "Student " << i + 1 << " marks: ";
        cin >> marks;
        cin.ignore();
        ofs << name << ',' << marks << '\n';
    }
    ofs.close();

    ifstream ifs(filename);
    if (!ifs)
    {
        cerr << "Cannot open file to read.\n";
        return 1;
    }

    cout << "\nSaved students:\n";
    string line;
    while (getline(ifs, line))
    {
```

```
        auto pos = line.find(',');
        if (pos != string::npos)
        {
            string name = line.substr(0, pos);
            string marks = line.substr(pos + 1);
            cout << "Name: " << name << "  Marks: " << marks << '\n';
        }
    }
    ifs.close();
    return 0;
}

// Output:

// Enter name and marks for 5 students:
// Student 1 name: Suresh
// Student 1 marks: 89
// Student 2 name: Mahesh
// Student 2 marks: 82
// Student 3 name: Khushi
// Student 3 marks: 82
// Student 4 name: Umang
// Student 4 marks: 98
// Student 5 name: Yash
// Student 5 marks: 78

// Saved students:
// Name: Suresh  Marks: 89
// Name: Mahesh  Marks: 82
// Name: Khushi  Marks: 82
// Name: Umang   Marks: 98
// Name: Yash   Marks: 78
```

**// 15) Write a C++ program that displays a student object read from a file**

**// backwards.**

```
#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

int main()
{
   ifstream fin("student.txt");
   string name;
   int marks;

   vector<string> data;

   while (fin >> name >> marks)
   {
      data.push_back(name + " " + to_string(marks));
   }
   fin.close();

   cout << "\n--- Displaying students backwards ---\n";
   for (int i = data.size() - 1; i >= 0; i--)
   {
      cout << data[i] << endl;
   }

   return 0;
}

// Output:

// --- Displaying students backwards ---
// Khushi 83
// Usha 83
```

// Nilesh 33

// Mukesh 87

// Meera 78

**// 16) A file contains name and phone numbers. Name contains only one word and**

**// names and telephone numbers are separated by white spaces. Write a**

**// program which will include a menu driven functionality for the above file**

**// (a) Display 2 column output with names left justified and phone numbers**

**// right justified.**

**// Name Phone**

**// Ram 56789125**

**// Shyam 34565432**

**// Rahim 23456789**

**// Joseph 67213148**

**// (b) Determine the telephone number of a person.**

**// (c) Determine the name if the telephone number is known.**

**// (d) Update the telephone number.**

**// (e) Prefix all the telephone numbers by 2.**

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <string>
using namespace std;

void display()
{
   ifstream fin("data.txt");
   if (!fin)
   {
      cout << "File not found!\n";
      return;
   }

   string name, phone;
```

```
    cout << left << setw(15) << "Name" << right << setw(15) << "Phone" << endl;
    cout << "----------------------------------------\n";

    while (fin >> name >> phone)
    {
        cout << left << setw(15) << name << right << setw(15) << phone << endl;
    }

    fin.close();
}

void searchPhoneByName()
{
    ifstream fin("data.txt");
    string name, phone, key;

    cout << "Enter name: ";
    cin >> key;

    bool found = false;

    while (fin >> name >> phone)
    {
        if (name == key)
        {
            cout << "Phone Number: " << phone << endl;
            found = true;
            break;
        }
    }

    if (!found)
        cout << "Name not found.\n";
    fin.close();
}
```

```
void searchNameByPhone()
{
    ifstream fin("data.txt");
    string name, phone, key;

    cout << "Enter phone number: ";
    cin >> key;

    bool found = false;

    while (fin >> name >> phone)
    {
        if (phone == key)
        {
            cout << "Name: " << name << endl;
            found = true;
            break;
        }
    }

    if (!found)
        cout << "Phone not found.\n";
    fin.close();
}

void updatePhone()
{
    ifstream fin("data.txt");
    ofstream fout("temp.txt");

    string name, phone, key, newPhone;
    cout << "Enter name to update phone: ";
    cin >> key;

    bool updated = false;
```

```
    while (fin >> name >> phone)
    {
        if (name == key)
        {
            cout << "Enter new phone: ";
            cin >> newPhone;
            fout << name << " " << newPhone << endl;
            updated = true;
        }
        else
        {
            fout << name << " " << phone << endl;
        }
    }

    fin.close();
    fout.close();

    remove("data.txt");
    rename("temp.txt", "data.txt");

    if (updated)
        cout << "Updated successfully!\n";
    else
        cout << "Name not found!\n";
}

void prefixPhone()
{
    ifstream fin("data.txt");
    ofstream fout("temp.txt");

    string name, phone;

    while (fin >> name >> phone)
```

```
    {
        fout << name << " " << "2" + phone << endl;
    }

    fin.close();
    fout.close();

    remove("data.txt");
    rename("temp.txt", "data.txt");

    cout << "All phone numbers prefixed with '2'.\n";
}

int main()
{
    int choice;

    do
    {
        cout << "\n--- PHONE DIRECTORY MENU ---\n";
        cout << "1. Display (Names left, Phones right)\n";
        cout << "2. Search Phone by Name\n";
        cout << "3. Search Name by Phone\n";
        cout << "4. Update Phone Number\n";
        cout << "5. Prefix all Phone Numbers with 2\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            display();
            break;
        case 2:
            searchPhoneByName();
```

```
            break;
        case 3:
            searchNameByPhone();
            break;
        case 4:
            updatePhone();
            break;
        case 5:
            prefixPhone();
            break;
        case 6:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice!\n";
        }

    } while (choice != 6);

    return 0;
}

// Output:

// --- PHONE DIRECTORY MENU ---
// 1. Display (Names left, Phones right)
// 2. Search Phone by Name
// 3. Search Name by Phone
// 4. Update Phone Number
// 5. Prefix all Phone Numbers with 2
// 6. Exit
// Enter your choice: 1
// Name                    Phone
// ----------------------------------------
// Name                    Phone
// Ram                 56789125
```

// Shyam 34565432

// Rahim 23456789

// Joseph 67213148

// --- PHONE DIRECTORY MENU ---

// 1. Display (Names left, Phones right)

// 2. Search Phone by Name

// 3. Search Name by Phone

// 4. Update Phone Number

// 5. Prefix all Phone Numbers with 2

// 6. Exit

// Enter your choice: 2 Ram

// Enter name: Phone Number: 56789125

// --- PHONE DIRECTORY MENU ---

// 1. Display (Names left, Phones right)

// 2. Search Phone by Name

// 3. Search Name by Phone

// 4. Update Phone Number

// 5. Prefix all Phone Numbers with 2

// 6. Exit

// Enter your choice: 6

// Exiting...

**// 17) Use an Employee Class to write records of employee to a file. Include a menu**

**// that will allow the user to select any of the following features**

**// a. Add a new record.**

**// b. Modify an existing record.**

**// c. Retrieve and display an entire record for a given name.**

**// d. Generate a complete list of all names, addresses and telephone numbers.**

**// e. End of the computation**.

```
#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;
```

```
class Employee
{
public:
    char name[30];
    char address[50];
    char phone[15];

    void readData()
    {
        cout << "Enter Name: ";
        cin >> name;
        cout << "Enter Address: ";
        cin >> address;
        cout << "Enter Phone: ";
        cin >> phone;
    }

    void showData()
    {
        cout << "Name: " << name << endl;
        cout << "Address: " << address << endl;
        cout << "Phone: " << phone << endl;
        cout << "----------------------------\n";
    }
};

int main()
{
    Employee emp;
    fstream file;
    int choice;
    char searchName[30];

    while (true)
    {
```

```
cout << "\n--- EMPLOYEE FILE MENU ---\n";
cout << "1. Add New Record\n";
cout << "2. Modify Existing Record\n";
cout << "3. Display Record by Name\n";
cout << "4. Display All Records\n";
cout << "5. Exit\n";
cout << "Enter choice: ";
cin >> choice;

if (choice == 5)
    break;

switch (choice)
{

case 1: // Add record
    file.open("employee.dat", ios::binary | ios::app);
    emp.readData();
    file.write((char *)&emp, sizeof(emp));
    file.close();
    break;

case 2: // Modify record
    cout << "Enter name to modify: ";
    cin >> searchName;

    file.open("employee.dat", ios::binary | ios::in | ios::out);
    while (file.read((char *)&emp, sizeof(emp)))
    {
        if (strcmp(emp.name, searchName) == 0)
        {
            cout << "Enter new details:\n";
            emp.readData();
            file.seekp(-sizeof(emp), ios::cur);
            file.write((char *)&emp, sizeof(emp));
            break;
```

```
            }
        }
        file.close();
        break;

    case 3: // Display specific record
        cout << "Enter name to search: ";
        cin >> searchName;

        file.open("employee.dat", ios::binary | ios::in);
        while (file.read((char *)&emp, sizeof(emp)))
        {
            if (strcmp(emp.name, searchName) == 0)
            {
                emp.showData();
                break;
            }
        }
        file.close();
        break;

    case 4: // Display all records
        file.open("employee.dat", ios::binary | ios::in);
        while (file.read((char *)&emp, sizeof(emp)))
        {
            emp.showData();
        }
        file.close();
        break;

    default:
        cout << "Invalid choice!\n";
    }
}

return 0;
```

```
}

// Output:

// --- EMPLOYEE FILE MENU ---
// 1. Add New Record
// 2. Modify Existing Record
// 3. Display Record by Name
// 4. Display All Records
// 5. Exit
// Enter choice: 1
// Enter Name: Khushi
// Enter Address: nikol
// Enter Phone: 8283

// --- EMPLOYEE FILE MENU ---
// 1. Add New Record
// 2. Modify Existing Record
// 3. Display Record by Name
// 4. Display All Records
// 5. Exit
// Enter choice: 4
// Name: Khushi
// Address: nikol
// Phone: 8283
// ----------------------------

// --- EMPLOYEE FILE MENU ---
// 1. Add New Record
// 2. Modify Existing Record
// 3. Display Record by Name
// 4. Display All Records
// 5. Exit
// Enter choice: 5
```

**// 18) Write a program that stores and displays the records of the customer from a**

**// file the following information for account of the customer is to be stored.**

**// Account no, account type, name, old balance, new balance, last payment,**

**// date of last payment. Also display the current account status by comparing**

**// OBJECT ORIENTED CONCEPTS AND PROGRAMING**

**// ASSIGNMENT-III**

**// current payment and previous balance. Also calculate the current balance by**

**// subtracting the current payment from the previous balance.**

```
#include <iostream>
#include <fstream>
using namespace std;

struct Customer
{
   int accNo;
   char accType[10];
   char name[20];
   float oldBalance;
   float lastPayment;
   float newBalance;
   char lastPayDate[15];
};

int main()
{
   Customer c;

   // -------- WRITE RECORD --------
   ofstream fout("customer.txt");
   if (!fout)
   {
      cout << "Error opening file!";
      return 0;
   }
```

```
cout << "Enter Account No: ";
cin >> c.accNo;

cout << "Enter Account Type: ";
cin >> c.accType;

cout << "Enter Name: ";
cin >> c.name;

cout << "Enter Old Balance: ";
cin >> c.oldBalance;

cout << "Enter Last Payment: ";
cin >> c.lastPayment;

cout << "Enter Date of Last Payment: ";
cin >> c.lastPayDate;

// Calculate new balance
c.newBalance = c.oldBalance - c.lastPayment;

// Write to file
fout << c.accNo << " "
    << c.accType << " "
    << c.name << " "
    << c.oldBalance << " "
    << c.lastPayment << " "
    << c.newBalance << " "
    << c.lastPayDate;

fout.close();

// -------- READ & DISPLAY --------
ifstream fin("customer.txt");
if (!fin)
{
```

```
        cout << "Error reading file!";
        return 0;
    }

    fin >> c.accNo >> c.accType >> c.name >> c.oldBalance >> c.lastPayment >> c.newBalance >> c.lastPayDate;

    fin.close();

    cout << "\n---- CUSTOMER DETAILS ----\n";
    cout << "Account No: " << c.accNo << endl;
    cout << "Account Type: " << c.accType << endl;
    cout << "Name: " << c.name << endl;
    cout << "Old Balance: " << c.oldBalance << endl;
    cout << "Last Payment: " << c.lastPayment << endl;
    cout << "Date of Last Payment: " << c.lastPayDate << endl;
    cout << "New Balance: " << c.newBalance << endl;

    // -------- STATUS CHECK --------
    if (c.lastPayment >= c.oldBalance)
        cout << "Account Status: CLEAR\n";
    else
        cout << "Account Status: PENDING\n";

    return 0;
}

// Output:
// Enter Account No: 833
// Enter Account Type: Saving
// Enter Name: Manoj
// Enter Old Balance: 80000
// Enter Last Payment: 823
// Enter Date of Last Payment: 20th may

// ---- CUSTOMER DETAILS ----
// Account No: 833
```

```
// Account Type: Saving
// Name: Manoj
// Old Balance: 80000
// Last Payment: 823
// Date of Last Payment: 20th
// New Balance: 79177
// Account Status: PENDING
```

**// Console I/O, Manipulators, Namespace and STL:**

**// 19) Design a manipulator to provide the following output specifications for**

**// printing float values**

**// (i) 5 column width**

**// (ii) Right justified**

**// (iii) 2 digits precision**

**// (iv) Filling unused spaces with +**

```
#include <iostream>
#include <iomanip>
using namespace std;

ostream &myformat(ostream &out)
{
   out << fixed << setprecision(2) << setw(5) << setfill('+') << right;
   return out;
}

int main()
{
   float a = 12.3456, b = 3.1, c = 89.999;

   cout << myformat << a << endl;
   cout << myformat << b << endl;
   cout << myformat << c << endl;

   return 0;
}
```

```
// Output:
// 12.35
// +3.10
// 90.00
```

**// 20) Define a class marksheet. The class should contain a function**
**// PrintMarkSheet such that it prints the marksheet of a given student with**
**// three subject names and five marks for each subject. Define manipulators for**
**// displaying headings and footnotes. The function should display marksheet**
**// with respective headings and class. The marks should be aligned under the**
**// headings (Use either ios functions or manipulators).**

```
#include <iostream>
#include <iomanip>
using namespace std;

class MarkSheet
{
public:
   void PrintMarkSheet()
   {
      string name;
      string sub1, sub2, sub3;
      int m1[5], m2[5], m3[5];

      cout << "Enter student name: ";
      getline(cin, name);

      cout << "Enter Subject 1 name: ";
      getline(cin, sub1);
      cout << "Enter 5 marks for " << sub1 << ": ";
      for (int i = 0; i < 5; i++)
         cin >> m1[i];

      cin.ignore();
      cout << "Enter Subject 2 name: ";
```

```
getline(cin, sub2);
cout << "Enter 5 marks for " << sub2 << ": ";
for (int i = 0; i < 5; i++)
    cin >> m2[i];

cin.ignore();
cout << "Enter Subject 3 name: ";
getline(cin, sub3);
cout << "Enter 5 marks for " << sub3 << ": ";
for (int i = 0; i < 5; i++)
    cin >> m3[i];

cout << "\n\n========== MARKSHEET ==========\n";
cout << left << setw(20) << "Student Name: " << name << endl;
cout << "-------------------------------------------\n";

// headings
cout << left << setw(15) << "Subject"
     << setw(8) << "M1"
     << setw(8) << "M2"
     << setw(8) << "M3"
     << setw(8) << "M4"
     << setw(8) << "M5" << endl;

cout << "-------------------------------------------\n";

// subject 1 marks
cout << left << setw(15) << sub1;
for (int i = 0; i < 5; i++)
    cout << setw(8) << m1[i];
cout << endl;

// subject 2 marks
cout << left << setw(15) << sub2;
for (int i = 0; i < 5; i++)
    cout << setw(8) << m2[i];
```

```
        cout << endl;

        // subject 3 marks
        cout << left << setw(15) << sub3;
        for (int i = 0; i < 5; i++)
            cout << setw(8) << m3[i];
        cout << endl;

        cout << "-----------------------------------------\n";
        cout << "=============== END ===============\n";
    }
};

int main()
{
    MarkSheet m;
    m.PrintMarkSheet();
    return 0;
}
// Output:

// Enter student name: Moksh
// Enter Subject 1 name: Science
// Enter 5 marks for Science: 83
// 23 53 53 53
// Enter Subject 2 name: Maths
// Enter 5 marks for Maths: 82
// 93
// 53
// 85
// 35
// Enter Subject 3 name: English
// Enter 5 marks for English: 88
// 98
// 78
// 99
```

```
// 90

// ========== MARKSHEET ==========
// Student Name:      Moksh
// -------------------------------------------
// Subject       M1     M2     M3     M4     M5
// -------------------------------------------
// Science       83     23     53     53     53
// Maths         82     93     53     85     35
// English       88     98     78     99     90
// -------------------------------------------
// =============== END ===============
```

**// 21) WAP to create namespace having function for total_marks. Show its use in**

**// class 'marks' of students, display total marks of subjects using namespace.**

```
#include <iostream>
using namespace std;

namespace studentNS
{
   int total_marks(int a, int b, int c)
   {
      return a + b + c;
   }
}

class Marks
{
public:
   int s1, s2, s3;

   void input()
   {
      cout << "Enter marks of 3 subjects: ";
```

```
        cin >> s1 >> s2 >> s3;
    }

    void showTotal()
    {
        cout << "Total Marks = "
            << studentNS::total_marks(s1, s2, s3);
    }
};

int main()
{
    Marks m;
    m.input();
    m.showTotal();
    return 0;
}
// Output:
// Enter marks of 3 subjects: 83 83 89
// Total Marks = 255
```

**// 22) WAP to define a vector and use it for student class to store and display**

**// information about student (Use STL).**

```
#include <iostream>
#include <vector>
using namespace std;

class Student
{
    string name;
    int age;

public:
    void read()
    {
```

```
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter age : ";
        cin >> age;
    }
    void show()
    {
        cout << name << "\t" << age << endl;
    }
};

int main()
{
    vector<Student> v;
    int n;

    cout << "How many students? ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        Student s;
        s.read();
        v.push_back(s);
    }

    cout << "\n--- Student Details ---\n";
    for (auto &x : v)
        x.show();

    return 0;
}
// Output:
// How many students? 2
// Enter name: Moksh
// Enter age : 25
```

```
// Enter name: Umang
// Enter age : 48

// --- Student Details ---
// Moksh   25
// Umang   48
```